



LAB WORKBOOK

25CS2101

**FRONT END DEVELOPMENT FRAMEWORKS FOR WEB
APPLICATIONS LAB WORKBOOK**

TEAM FRONT END DEVELOPMENT FRAMEWORKS FOR WEB APPLICATIONS

K L UNIVERSITY | VADDESWARAM

LAB WORKBOOK

25CS2101

FRONT END DEVELOPMENT FRAMEWORKS FOR WEB APPLICATIONS LAB WORKBOOK

STUDENT NAME	
STUDENT ID	
YEAR	
SEMESTER	
SECTION	
FACULTY NAME	

DEPARTMENT VISION AND MISSION

Vision

To be a Department of International Repute through Continuous Research, Innovation and Industry Led Curriculum.

Mission

To Impart Quality Education with Social Consciousness and make them globally competent.

Program Educational Objectives

1. Practice engineering in a broad range of industrial, societal and real-world applications.
2. Pursue advanced education, research and development, by adapting creative and innovative practices in their professional careers.
3. Conduct themselves in a responsible, professional, and ethical manner.
4. Participate as leaders in their fields of expertise and in activities that support service and economic development throughout the world.

SYLLABUS:

CO-1: Git, Version Control, and Introduction to HTML/CSS

Introduction to Git and Version Control, Git Installation and Configuration, Git Life Cycle Repository Creation, Remote Repository basic Operations, Branching and Merging, Conflict Resolution, Git History Management, .gitignore, and Git Workflow. Introduction to HTML5 structure, Semantic HTML5 tags, Forms and input types, Media elements (audio, video, images), Advanced HTML5 elements (section, aside, nav, header, footer), HTML5 form enhancements (placeholder, required, autofocus, pattern).

CO-2: CSS3, and Core JavaScript

Introduction to CSS, CSS Syntax, Types of CSS, Selectors, Box Model, Positioning Techniques, Flexbox and Grid Layouts, Responsive Web Design Principles, Media Types and Queries, CSS Transitions, Transformations, and Keyframe Animations. Introduction to JavaScript, Syntax, Data Types, Operators, Functions, Control Structures, Objects, Arrays, Sets, Maps, DOM Manipulation, Event Handling, ES6+ Features, Promises, Async/Await, Fetch API, Form Validation, Error Handling, Local and Session Storage, Modular JavaScript, Classes and Objects, JSON Processing, and Debugging Tools..

CO-3: React Fundamentals and UI Development

Introduction to React, JSX, Functional Components, Props and State, Event Handling, Conditional and List Rendering, React Hooks (useState, useEffect), Forms, Component Communication, React Router, Application Structure, Reusable Components, Atomic Design, Redux Toolkit, MaterialUI, Tailwind CSS, and Deployment with Vite.

CO-4: Advanced React, Testing, and Backend with Node.js/Express

Advanced React Concepts (Context API, Custom Hooks, Lazy Loading, Error Boundaries), React Testing with Jest and React Testing Library, Introduction to Node.js, npm and HTTP Server Development, Express.js Setup and Routing, RESTful API Development (GET/POST), Middleware, Security Practices, Input Validation, and Data Sanitization. CO-5 Introduction to Next.js, React vs Next.js, File-Based and Dynamic Routing, API Routes, SSG and SSR, Data Fetching, Image Optimization, SEO, Middleware and NextAuth.js Authentication, MongoDB and Mongoose Integration, Environment Variables, Tailwind CSS, Deployment on Vercel, and Full-Stack Capstone Project Development.

PROGRAM OUTCOMES		
PO	Graduate Attributes	Program Outcome Description
1	Engineering Knowledge	To impart mathematics, science, & engineering knowledge to develop skills to solve complex engineering problems.
2	Problem Analysis	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3	Design/ development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4	Conduct investigations of complex problems	An ability to use research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.
5	Modern tool usage	Ability to create, select and apply appropriate techniques, resources and modern engineering activities, while understanding its limitations.
6	The engineer and society	Ability to apply reasoning and the contextual knowledge to assess social & health safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practices.
7	Environment and sustainability	Ability to demonstrate the engineering knowledge to find solutions to contemporary issues by understanding their impact on societal and environmental contexts, towards sustainable development
8	Ethics	An ability to apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
9	Individual and teamwork	To inculcate abilities to be able to act as a leader as well as team player effectively in multi-disciplinary settings
10	Communication	To develop oral and written communication skills to articulate the complex engineering activities with the engineering community and society effectively through reports and design documentation, make effective presentations, and give and receive clear instructions.
11	Project management and finance	To develop working knowledge and understanding of the engineering and management principles to manage projects in multi-disciplinary environments.
12	Lifelong learning	To inculcate the habit of constant knowledge upgrading habit to meet the ever-changing technology and industry needs.
PROGRAM SPECIFIC OUTCOMES		
PSO1	An ability to design and develop software projects as well as to analyze and test user requirements.	
PSO2	Working knowledge on emerging technologies as per the industry requirements	

LAB EXPERIMENTS

Exp. No.	Experiment Name	Page No.
LAB – 1	➔ Create Git Repository and perform Git Operations (Push, Fetch, Merge, Pull, Reset, Revert, Clone and Fork)	00
LAB – 2	➔ Create a Git Repository and Develop Features Using Multiple Branches, Merge Feature Branches into the Main Branch	00
LAB – 3	➔ Deploy Static Web Application into git Pages using git workslows	00
LAB – 4	➔ Design the homepage of an e-commerce website using CSS Flexbox, Grid, and animation techniques.	00
LAB – 5	➔ Develop a responsive Student Admission Form with CSS Grid layout that works seamlessly across desktop, tablet, and mobile devices.	00
LAB – 6	➔ Develop a web-based Student Result Management System using html,css and JavaScript.	00
LAB – 7	➔ Develop a Weather Information Dashboard using modern JavaScript features. Fetch weather data from an API using Fetch API	00
LAB – 8	➔ Develop a React application with Login, Dashboard, and Profile modules.	00
LAB – 9	➔ Building a Course Management Dashboard Using React, Redux Toolkit, and Material UI	00
LAB – 10	➔ Designing a University Event Management Portal Using Atomic Design, Material UI, and Tailwind CSS	00
LAB – 11	➔ Create React based application authentication with a Node.js Express backend.	00
LAB – 12	➔ Developing a Secure Student Portal Using Context API and Express Middleware	00

2026-27 ODD SEMESTER FRONTEND DEVELOPMENT FRAMEWORKS FOR WEB APPLICATIONS EVALUATION

Lab No.	Date of Evaluation	Implementation (20M)	Output (20M)	Viva Voce (10M)	Total (50M)	Faculty Signature
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 1 → Create Git Repository and perform Git Operations (Push, Fetch, Merge, Pull, Reset, Revert, Clone and Fork)

Prerequisites:

- Have **Git installed** on your system.
- Understand **basic Git commands** (init, add, commit, status, log).
- Have a **Git repository** initialized (git init or a cloned repo).
- Understand **how to edit and save files** in a text/code editor (like VS Code, Sublime, or Vim).
- Know how to use the **command line (CLI)** for Git operations.

Scenario: Managing a Remote Repository for a Web Project

You have completed the initial version of a web application on your local system. Now, you need to store the project on GitHub so that it can be backed up and shared with your team.

After uploading the project, you will perform common Git remote operations such as **Push, Fetch, Merge, Pull, Reset, Revert, Clone, and Fork** to understand how developers collaborate using local and remote repositories.

Steps:

1. Create a local Git repository.
2. Create a GitHub repository and connect it to the local repository.
3. Push the local project to GitHub.
4. Make changes in the remote repository and fetch them into the local repository.
5. Merge the fetched changes into the local branch.
6. Perform a pull operation to update the local repository.
7. Use Reset to undo local commits.
8. Use Revert to undo a committed change while preserving history.
9. Clone the remote repository into another local folder.
10. Fork a public GitHub repository into your own GitHub account and clone the fork.

Project Structure (Example):

Project Structure (Before Push)

```
git-remote-demo/  
|  
├── index.html  
├── style.css  
├── script.js  
└── README.md
```

Project Structure (After Remote Operations)

```
git-remote-demo/  
|  
├── index.html  
├── style.css  
├── script.js  
├── README.md  
└── images/  
    └── logo.png
```


VIVA QUESTIONS:

1. What is the difference between a local repository and a remote repository?
2. What is the purpose of the git push command?
3. What is the difference between git fetch and git pull?
4. What is the difference between Fork and Clone?
5. Why do developers use Fork instead of Clone when contributing to open-source projects?

(For Evaluator's use only)

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation</u> Marks Secured: _____ out of 50 Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:
---	--

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 2 → Create a Git Repository and Develop Features Using Multiple Branches, Merge Feature Branches into the Main Branch

Prerequisites:

1. Have Git installed on your system.
2. Understand basic Git commands (init, add, commit, status, log).
3. Know how to create and switch between branches.
4. Have a Git repository initialized (git init).
5. Understand how to edit and save files using a text editor (VS Code, Sublime Text, or Vim).
6. Basic knowledge of Command Line Interface (CLI).

Exercise: (Scenario)

Scenario: Developing Multiple Features for a Web Application

You are part of a web development team building a company website. To allow multiple developers to work independently, each new feature is developed in a separate Git branch.

The main branch contains the stable version of the application. One developer creates a feature/header branch to design the website header, while another developer creates a feature/footer branch to develop the footer section. After completing the features, both branches are merged into the main branch to create the final version of the website.

This exercise demonstrates how Git enables parallel development using multiple branches and integrates completed features into the main branch.

Steps:

1. Initialize a Git repository and create the base project.
2. Create the **feature/header** branch and develop the header section.
3. Commit the changes in the **feature/header** branch.
4. Switch to the **main** branch.
5. Create the **feature/footer** branch and develop the footer section.
6. Commit the changes in the **feature/footer** branch.
7. Switch back to the **main** branch.
8. Merge the **feature/header** branch into **main**.

9. Merge the **feature/footer** branch into **main**.
10. Verify the merged project and commit history.

Project Structure (Initial)

git-multi-branch-demo/

```
├── index.html
├── style.css
└── README.md
```

Project Structure (After Merging)

git-multi-branch-demo/

```
├── index.html
├── style.css
├── header.html
├── footer.html
└── README.md
```


VIVA QUESTIONS:

1. Why do developers create separate branches for new features?
2. What is the purpose of the main branch?
3. How do you create a new branch in Git?
4. How do you switch between branches?
5. What command is used to merge a feature branch into the main branch?

(For Evaluator's use only)

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation</u> Marks Secured: _____ out of 50 Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:
---	--

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 3 → Deploy Static Web Application into git Pages using git workslows

Prerequisites

- Have Git installed on your system.
- Have a GitHub account.
- Understand basic Git commands (init, add, commit, push).
- Have a Git repository connected to GitHub.
- Know the basics of HTML and CSS.
- Understand how to use a text editor such as VS Code.
- Basic knowledge of GitHub Actions (Workflows).

Scenario: Deploying a Static Website Using GitHub Pages

You have completed a static website for your organization and stored the project in a GitHub repository. Instead of manually uploading the website to a hosting server after every update, you decide to automate the deployment process using **GitHub Actions**.

Whenever new changes are pushed to the **main** branch, GitHub Actions automatically builds and deploys the website to **GitHub Pages**, ensuring that the latest version is always available online.

This exercise demonstrates how to configure GitHub Pages and create a GitHub Actions workflow for continuous deployment of a static web application.

Steps

1. Create a Git repository and develop a static web application.
2. Push the project to a GitHub repository.
3. Configure GitHub Pages to deploy using **GitHub Actions**.
4. Create the GitHub Actions workflow file.
5. Commit and push the workflow to GitHub.
6. Verify that the workflow executes successfully.
7. Access the deployed website using the GitHub Pages URL.
8. Modify the website and push the changes to trigger automatic deployment.

Project Structure (Before Deployment)

static-web-app/

```
|
|— index.html
|— style.css
|— script.js
└— README.md
```

Project Structure (After Adding Workflow)

static-web-app/

```
|
|— .github/
|   └— workflows/
|       └— deploy.yml
|
|— index.html
|— style.css
|— script.js
└— README.md
```


VIVA QUESTIONS:

1. What is GitHub Pages?
2. What is GitHub Actions?
3. What is a workflow in GitHub Actions?
4. Where are GitHub workflow files stored?
5. What is the purpose of the .github/workflows directory?

(For Evaluator's use only)

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation</u> Marks Secured: _____ out of 50 Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:
---	--

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 4 ➔ Clone, Build & Run Frontend App in Docker

Prerequisites:

- Have a code editor such as Visual Studio Code installed.
- Understand the basic structure of HTML5.
- Know the fundamentals of CSS syntax and selectors.
- Familiarity with CSS Flexbox and Grid layouts.
- Basic knowledge of CSS transitions, transformations, and keyframe animations.
- Have a modern web browser (Chrome, Firefox, or Edge) for testing.

Exercise (Scenario):

Scenario: Designing an E-Commerce Website Homepage

You have been assigned the role of a front-end developer for an online shopping platform. The company requires an attractive and responsive homepage that provides a professional user experience across different screen sizes.

Your task is to design the homepage using CSS Flexbox for navigation and alignment, CSS Grid for organizing product sections, and CSS animations to enhance the visual appeal of banners, buttons, and product cards. The final webpage should be responsive, interactive, and visually appealing.

Steps

- Create a project folder for the e-commerce website.
- Create the required HTML and CSS files.
- Design the navigation bar using CSS Flexbox.
- Create a hero/banner section with an animated call-to-action button.
- Design the product section using CSS Grid.
- Add hover effects and transitions to product cards.
- Apply keyframe animations to banners or promotional content.
- Make the webpage responsive using media queries.
- Test the webpage in a web browser.

VIVA QUESTIONS

1. What is CSS Flexbox?
2. What are the advantages of using Flexbox?
3. What is CSS Grid?
4. How is CSS Grid different from Flexbox?
5. What is the purpose of the display: flex property?

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 5 ➔ Develop a responsive Student Admission Form with CSS Grid layout that works seamlessly across desktop, tablet, and mobile devices.

Prerequisites:

1. Have a code editor such as Visual Studio Code installed.
2. Understand the basic structure of HTML5.
3. Know the fundamentals of CSS syntax and selectors.
4. Familiarity with CSS Flexbox and Grid layouts.
5. Basic knowledge of CSS transitions, transformations, and keyframe animations.
6. Have a modern web browser (Chrome, Firefox, or Edge) for testing.

Exercise (Scenario):

Scenario: Designing an E-Commerce Website Homepage

You have been assigned the role of a front-end developer for an online shopping platform. The company requires an attractive and responsive homepage that provides a professional user experience across different screen sizes.

Your task is to design the homepage using CSS Flexbox for navigation and alignment, CSS Grid for organizing product sections, and CSS animations to enhance the visual appeal of banners, buttons, and product cards. The final webpage should be responsive, interactive, and visually appealing.

Steps:

1. Create a project folder for the e-commerce website.
2. Create the required HTML and CSS files.
3. Design the navigation bar using CSS Flexbox.
4. Create a hero/banner section with an animated call-to-action button.
5. Design the product section using CSS Grid.

Viva Questions:

1. What is CSS Flexbox?
2. What are the advantages of using Flexbox?
3. What is CSS Grid?
4. How is CSS Grid different from Flexbox?
5. What is the purpose of the display: flex property?

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 6 ➔ Develop a web-based Student Result Management System using html,css and JavaScript.

Prerequisites:

1. Have a code editor such as Visual Studio Code installed.
2. Understand the basic structure of HTML5.
3. Know the fundamentals of CSS syntax, selectors, and layouts.
4. Familiarity with JavaScript variables, functions, arrays, objects, and event handling.
5. Basic knowledge of DOM manipulation and form validation.
6. Have a modern web browser (Chrome, Firefox, or Edge) for testing.

Exercise (Scenario):

Scenario: Developing a Student Result Management System

A college wants to automate the process of displaying students' examination results through a web application. The system should allow users to enter student details and marks for different subjects, automatically calculate the total marks, percentage, grade, and pass/fail status, and display the results in a well-formatted table.

As a front-end developer, your task is to build a responsive web-based Student Result Management System using HTML, CSS, and JavaScript without using any backend technology.

Steps:

1. Create a project folder for the Student Result Management System.
2. Create the required HTML, CSS, and JavaScript files.
3. Design a student result entry form using HTML.
4. Style the application using CSS.
5. Validate user inputs using JavaScript.
6. Calculate the total marks, percentage, grade, and result status.
7. Display the student result dynamically in a table.
8. Provide options to clear the form and enter another student's details.
9. Test the application using different input values

Viva Questions:

1. What is DOM manipulation in JavaScript?
2. What is the purpose of form validation?
3. What is the difference between innerHTML and textContent?
4. Which JavaScript event is commonly used to submit a form?
5. What is the purpose of the addEventListener() method?

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 7 ➔ Develop a Weather Information Dashboard using modern JavaScript features. Fetch weather data from an API using Fetch API

Prerequisites:

- Have a code editor such as Visual Studio Code installed.
- Understand the basic structure of HTML5.
- Know the fundamentals of CSS syntax and responsive layouts.
- Familiarity with JavaScript ES6+ features such as **let**, **const**, arrow functions, template literals, destructuring, modules, async/await, and promises.
- Basic knowledge of DOM manipulation and event handling.
- Have an active internet connection to access a public Weather API (e.g., OpenWeatherMap API).

Exercise (Scenario):

Scenario:

A weather forecasting organization wants to provide users with real-time weather information through a responsive web application. The dashboard should allow users to search for any city and retrieve current weather details from an online Weather API.

As a front-end developer, your task is to build a **Weather Information Dashboard** using **HTML**, **CSS**, and **Modern JavaScript (ES6+)**. The application should fetch weather data using the **Fetch API**, process the received JSON data, and dynamically display weather information such as temperature, humidity, wind speed, weather conditions, and weather icons.

Steps:

1. Create a project folder for the Weather Information Dashboard.
2. Create the required HTML, CSS, and JavaScript files.
3. Design a responsive user interface with a search box and weather information panel.
4. Obtain an API key from a Weather API provider (e.g., OpenWeatherMap).
5. Use the Fetch API to retrieve weather data based on the entered city.
6. Parse the JSON response using JavaScript.
7. Display the weather information dynamically on the webpage.
8. Handle invalid city names and API errors gracefully.
9. Test the application using different city names.

Viva Questions:

1. What is the Fetch API?
2. What is a REST API?
3. What is JSON, and why is it used in web applications?
4. What is the purpose of the fetch() method?
5. What is the difference between Promises and Async/Await?

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 8 ➔ Develop a React application with Login, Dashboard, and Profile modules.**Prerequisites:**

1. Have Node.js and npm installed on your system.
2. Have Visual Studio Code or any modern code editor installed.
3. Understand the fundamentals of HTML, CSS, and JavaScript (ES6+).
4. Familiarity with React concepts such as components, JSX, props, state, and event handling.
5. Basic knowledge of React Hooks (useState, useEffect).
6. Basic understanding of React Router for navigation.
7. Have a modern web browser (Chrome, Firefox, or Edge) for testing.

Exercise (Scenario):**Scenario:**

A university wants to build a web-based Student Portal to provide students with a simple and interactive interface. The application should include a Login module for user authentication, a Dashboard module to display important information after successful login, and a Profile module to display the student's personal details.

As a front-end developer, your task is to build a responsive React application using reusable components, React Hooks, and React Router. The application should validate user credentials, navigate between pages, and display dynamic content without refreshing the browser.

Project Structure**student-portal/**

```

|
├── public/
|
├── src/
|   ├── components/
|   |   ├── Login.jsx
|   |   ├── Dashboard.jsx
|   |   └── Profile.jsx

```

```
| | └─ Navbar.jsx
| |
| └─ App.jsx
| └─ main.jsx
| └─ App.css
| └─ index.css
|
└─ package.json
└─ README.md
```

Steps:

- Create a new React application using Vite or Create React App.
- Install the required dependencies such as React Router.
- Create reusable React components for Login, Dashboard, and Profile.
- Configure routing between the modules.
- Design the Login page with form validation.
- Implement user authentication using React state.
- Display the Dashboard after successful login.
- Create the Profile page to display user information.
- Add a Logout option to return to the Login page.
- Test navigation and application functionality.

Viva Questions:

1. What is JSX?
2. What is the purpose of React components?
3. What is the difference between functional and class components?
4. What is the purpose of the useState() Hook?
5. What is the purpose of the useEffect() Hook?

Date of the Session: ____ / ____ / ____

Time of The Session: ____ to ____

LAB – 9 → Building a Course Management Dashboard Using React, Redux Toolkit, and Material UI

Prerequisites:

1. Have Node.js and npm installed on your system.
2. Have Visual Studio Code or any modern code editor installed.
3. Understand the fundamentals of HTML, CSS, and JavaScript (ES6+).
4. Familiarity with React concepts such as components, JSX, props, state, and event handling.
5. Basic knowledge of React Hooks (useState, useEffect).
6. Basic understanding of React Router for navigation.
7. Familiarity with Redux Toolkit for state management.
8. Basic knowledge of Material UI (MUI) components.
9. Have a modern web browser (Chrome, Firefox, or Edge) for testing.

Exercise (Scenario):

Scenario: Building a Course Management Dashboard

A university wants to digitize the management of its academic courses through a modern web application. The dashboard should allow administrators to view, add, update, and remove course details from a centralized interface. The application should provide an intuitive user experience with responsive layouts and reusable UI components.

As a front-end developer, your task is to build a **Course Management Dashboard** using **React**, **Redux Toolkit**, and **Material UI**. Use Redux Toolkit to manage the application state and Material UI components to create a responsive and professional user interface.

Steps:

1. Create a new React application using Vite or Create React App.
2. Install the required dependencies including React Router, Redux Toolkit, React-Redux, and Material UI.
3. Configure the Redux Store and create a Course Slice.
4. Design the Dashboard layout using Material UI components.
5. Create reusable components for displaying course information.
6. Implement Add Course functionality using Redux actions.

7. Implement Edit Course functionality.
8. Implement Delete Course functionality.
9. Display all available courses in a responsive table or card layout.
10. Test the application by performing CRUD operations.

VIVA QUESTIONS

1. What is Redux Toolkit?
2. Why is Redux Toolkit preferred over traditional Redux?
3. What is the purpose of the Redux Store?
4. What is a Slice in Redux Toolkit?
5. What are reducers in Redux?

(For Evaluator's use only)

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation</u> Marks Secured: _____ out of 50 Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:
--	---

Date of the Session: / /

Time of The Session:_____to_____

LAB – 10 ➔ Designing a University Event Management Portal Using Atomic Design, Material UI, and Tailwind CSS

Prerequisites:

- Have Node.js and npm installed on your system.
- Have Visual Studio Code or any modern code editor installed.
- Understand the fundamentals of HTML, CSS, and JavaScript (ES6+).
- Familiarity with React concepts such as components, JSX, props, state, and event handling.
- Basic knowledge of React Hooks (useState, useEffect).
- Basic understanding of component-based architecture.
- Familiarity with Material UI (MUI) components.
- Basic knowledge of Tailwind CSS utility classes.
- Have a modern web browser (Chrome, Firefox, or Edge) for testing.

Exercise (Scenario):

Scenario: Designing a University Event Management Portal

A university plans to develop a modern web application to manage academic and extracurricular events. The portal should enable students to browse upcoming events, register for events, and view event details, while administrators can manage event information through an intuitive dashboard.

As a front-end developer, your task is to design a University Event Management Portal using React, Atomic Design principles, Material UI, and Tailwind CSS. The application should follow a reusable component architecture, provide a responsive user interface, and ensure consistency across all pages.

Steps:

- Create a new React application using Vite or Create React App.
- Install the required dependencies including Material UI and Tailwind CSS.
- Configure Tailwind CSS in the React project.
- Design reusable UI components following the Atomic Design methodology.
- Create pages for Home, Events, Event Details, Registration, and About.
- Develop responsive layouts using Material UI Grid and Tailwind CSS.
- Implement navigation using React Router.
- Display event information using reusable cards and lists.

- Test the application across different screen sizes.
- Verify the responsiveness and component reusability.

LAB – 11 ➔ Create a React-Based Application Authentication with a Node.js Express Backend

Prerequisites:

- Have Node.js and npm installed on your system.
- Have Visual Studio Code or any modern code editor installed.
- Understand the fundamentals of HTML, CSS, and JavaScript (ES6+).
- Familiarity with React concepts such as components, JSX, props, state, and event handling.
- Basic knowledge of React Hooks (useState, useEffect).
- Basic understanding of Node.js and Express.js.
- Familiarity with REST APIs and JSON.
- Basic knowledge of MongoDB (or any NoSQL database).
- Have a modern web browser (Chrome, Firefox, or Edge) for testing.

Exercise (Scenario)

Scenario: Developing a Secure Student Portal Authentication System

A university is developing a Student Portal that allows students to securely access academic services. The application should provide user registration, login, authentication, protected dashboard access, and logout functionality. The frontend is developed using **React**, while the backend is implemented using **Node.js** and **Express.js**. User credentials are stored securely in a MongoDB database, and authenticated users can access protected resources.

As a full-stack developer, your task is to build a secure authentication system using **React**, **Node.js**, **Express.js**, and **MongoDB**. The application should communicate through REST APIs and validate user credentials before granting access to protected pages.

Steps

- Create separate React (frontend) and Node.js Express (backend) applications.
- Install the required dependencies for React and Express.
- Configure the Express server and MongoDB connection.
- Create a User model in MongoDB.
- Develop REST APIs for user registration and login.
- Implement JWT-based authentication for protected routes.
- Create Login and Registration pages in React.
- Connect the React application to the backend using Fetch API or Axios.

- Implement protected routes and logout functionality.
- Test the complete authentication workflow.

Viva Questions

- What is the role of Node.js in a web application?
- What is Express.js, and why is it used?
- What is MongoDB?
- What is a REST API?
- What is JWT (JSON Web Token)?

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation</u>
	Marks Secured: _____ out of 50
	Full Name of the Evaluator:
	Signature of the Evaluator Date of Evaluation:

Time of The Session: _____ to _____

LAB – 12 ➔ Developing a Secure Student Portal Using Context API and Express Middleware

Prerequisites:

- Have Node.js and npm installed on your system.
- Have Visual Studio Code or any modern code editor installed.
- Understand the fundamentals of HTML, CSS, and JavaScript (ES6+).
- Familiarity with React concepts such as components, JSX, props, state, and event handling.
- Basic knowledge of React Hooks (useState, useEffect).
- Familiarity with the React Context API for global state management.
- Basic understanding of Node.js and Express.js.
- Familiarity with REST APIs and JSON.
- Basic knowledge of MongoDB (or any NoSQL database).
- Have a modern web browser (Chrome, Firefox, or Edge) for testing.

Exercise (Scenario)

Scenario: Developing a Secure Student Portal

A university plans to build a secure Student Portal that allows students to register, log in, and access personalized academic information. The application should maintain the user's authentication status across multiple pages without prop drilling by using the React Context API. The backend, developed using Node.js and Express.js, should use Express Middleware to authenticate requests and protect sensitive resources.

As a full-stack developer, your task is to build a secure Student Portal using React, Context API, Node.js, Express.js, and MongoDB. The application should implement user authentication, manage global authentication state using Context API, and secure backend APIs using Express middleware.

Steps:

1. Create separate React (frontend) and Node.js Express (backend) applications.
2. Install the required dependencies for React and Express.
3. Configure the Express server and MongoDB connection.
4. Create a User model and authentication APIs.

5. Develop an Authentication Context using the React Context API.
6. Implement login, registration, and logout functionality.
7. Create protected routes using Context API.
8. Implement authentication middleware in Express.
9. Connect the React frontend with the backend using Fetch API or Axios.
10. Test the complete authentication workflow.

Viva Questions

1. What is the React Context API?
2. Why is the Context API used instead of prop drilling?
3. What is useContext() in React?
4. What is middleware in Express.js?
5. What is the purpose of authentication middleware?

(For Evaluator's use only)

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation</u> Marks Secured: _____ out of 50 Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:
---	--

